River Forecasting and Motion or Memory? Assignment 2

Rana Taqveem (Roll number: 25280030)
AI651 – Deep Learning for Space, Time and Graphs, Fall 2026

Abstract

This report answers both questions of Assignment 2. Question 1 proposes a forecasting system that predicts water levels one hour ahead at four river stations every ten minutes. It uses a factorized spatial–temporal attention encoder and a cross-attention decoder that reads announced reservoir releases. Data alignment, uncertainty bands and exceptional conditions are also covered. Question 2 tested whether a ConvLSTM trained on Moving MNIST learned motion or memorised appearance. The baseline beat the Blank and Copy-last references on every test video, but it was not consistent under mirror and rotation and degraded at higher speeds. Mixed-speed training helped on sped-up clips. The final symmetrised model is consistent under mirror and rotation, yet its predictions can still be wrong. Consistency alone does not prove correctness. Code: `https://github.com/rana-taqveem/dl4stg`.

1 Question 1: River Forecasting Proposal

1.1 Architecture diagram

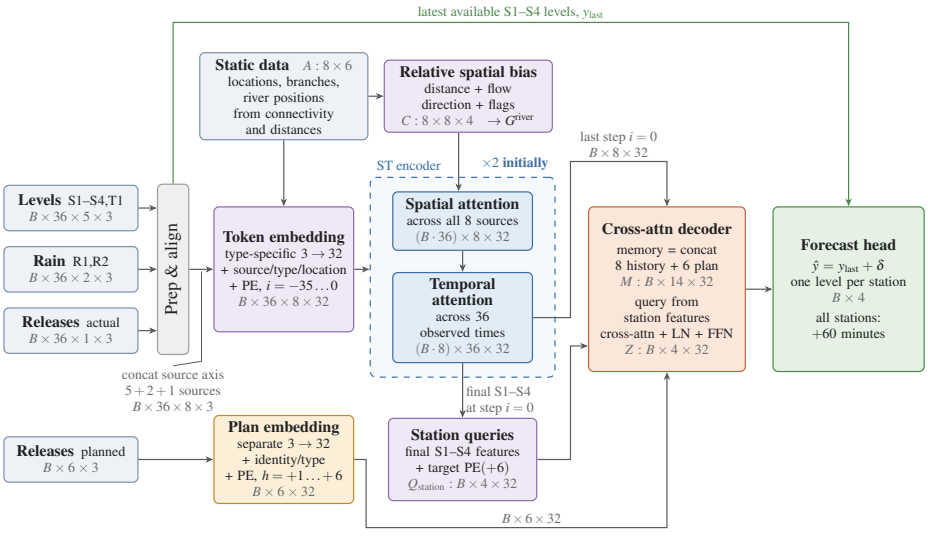


Figure Q1.1: Factorized-attention encoder and query decoder. The upper branch turns static metadata into source features and a river-aware attention bias. The historical branch keeps the source and time axes through spatial and temporal attention. At the last observed step, eight source summaries join six announced-plan tokens as decoder memory. Four station queries ask for levels 60 minutes ahead. The forecast head adds the predicted change to each station's latest level.

Reading the diagram. B is the batch size. Each forecast uses 36 past time steps, eight sources and 32 features per token. The output has shape $B \times 4$: one level for each main station. The source order is $(S_1, S_2, S_3, S_4, T_1, R_1, R_2, U)$, where U is the actual reservoir release.

Positional encoding (PE) tells the model when. We used fixed sine and cosine offsets: -35 to 0 for history, $+1$ to $+6$ for release plans, and $+6$ for the forecast target. One step is ten minutes. Source and river-location features tell the model where each reading comes from.

1.2 Assumptions

Supplied facts. The handout gave us five level stations, two rain gauges, past releases and announced release schedules, along with station locations, river connections and distances. The monitored tributary joins between S_1 and S_2, and the unmonitored one joins between S_2 and S_3. Stations are unevenly spaced, and travel times change. Future rainfall and future levels are not available. We made the assumptions below to fill the remaining gaps.

A1: historical coverage. We assumed about three years of usable records with several high-water events. Even this may cover rare floods poorly, so we started with a small shared model. With shorter records, we would reduce its width or depth. Before live use, we would compare it with persistence, which simply repeats the last reading.

A2: clocks, units and availability. We assumed all clocks can be converted to one timezone and every station is clearly identified. Water-level reference heights must stay fixed, or their changes must be recorded. Rainfall units and measurement intervals must also be known. Arrival times should be logged from deployment onward. If old arrival logs are missing, we would test with simulated delays, although such tests cannot prove live accuracy.

A3: history and travel time. We started with 36 ten-minute samples, which span 350 minutes. To check that this window is long enough, we used a simple distance test:

$$D_{\max}/c_{\text{slow}} + \tau_{\text{runoff}} \leq 350\,\text{min}.$$

Here $D_{\max}$ is the longest relevant river path, including the reservoir to S_4 if that distance is available. c_{slow} is a slow flood-wave speed, and τ_{runoff} is the delay before rain reaches the river. Neither value is supplied, so the authority must provide them, and this check is pending. The test is cautious, since a one-hour forecast may need less history. Validating longer windows would make the final choice.

A4: river relationships. We assumed water flowing downstream is the main physical influence. Each source's embedding describes its branch and position. A learned bias adds the river distance between two sources and whether one is upstream of the other. Attention can still see every source. Backwater, tides or downstream controls could change these relationships, so we would check for them.

A5: approximate stability. We assumed past patterns stay useful when conditions look similar. Attention weights depend on the current readings, so the model can focus on different sources and times as conditions change. We would monitor errors and review the model after channel changes or unusual floods.

A6: rainfall and tributary proxies. We assumed the two rain gauges and T_1 give useful but incomplete signals. Gauge locations alone do not tell us how rain becomes runoff, so the model learns how their readings relate to each forecast. We treated the T_1 level as a clue to tributary inflow, not as a flow measurement.

A7: operating plans. We assumed announcement and effective times identify the latest plan known when a forecast is issued. A plan describes intended releases, and actual releases may differ. If no plan covers an interval, we hold the latest release rate and flag it.

A8: computation and fallback. We assumed a small CPU server and a sensor database are available. We started with a one-hour limit on reading age; beyond it, the service switches to a labelled fallback. Validation would set the final limit. Below it, missing-data flags and reading ages remain model inputs.

1.3 Forecasting task and data specification

Every ten minutes, the system uses the information available at that moment to predict the level 60 minutes later at S_1–S_4. Let t_n be the issue time, with each step lasting ten minutes. The model reads

the grid $t_{n-35}, \ldots, t_n$ and targets $t_{n+6} = t_n + 60$ minutes:

$$\hat{\mathbf{y}}_{n+6} = \left(\hat{y}_{S_1,n+6}, \ldots, \hat{y}_{S_4,n+6}\right) \in \mathbb{R}^4.$$

It predicts all four levels in one pass, with no step-by-step rollout. Uncertainty bands are added after training, as described in Section 1.4.

Table Q1.1: Inputs and outputs of the forecasting task, with axes and time coverage.

Input/output	Dimensions and axes	Time coverage and meaning
Water levels X^h	$B \times 36 \times 5 \times 3$; batch, time, station, feature	Past grid up to t_n. Features: scaled level, fresh-observation mask, observation age. Station order: S_1–S_4, T_1.
Rainfall X^r	$B \times 36 \times 2 \times 3$; batch, time, gauge, feature	Past bins ending at t_n. Features: scaled ten-minute total, coverage flag, age. No future rainfall.
Actual releases X^u	$B \times 36 \times 1 \times 3$; batch, time, source, feature	Past release rate, observation mask and age, available by t_n. Aligned using the recorded rate and interval definitions.
Release plan K	$B \times 6 \times 3$; batch, future bin, feature	Bins $(t_n, t_{n+1}], \ldots, (t_{n+5}, t_{n+6}]$. Planned or held rate, known-plan coverage, lead time. Only announcements available by t_n.
Static metadata A	8×6; source, feature	Latitude, longitude, main-river position, coordinates-known flag, river-position-known flag and branch code. Unknown values keep their flags.
River pair features C	$8 \times 8 \times 4$; query source, key source, feature	Along-river distance, distance-known flag, upstream/downstream code and relation-known flag. Derived from the supplied connectivity and distances; no adjacency mask.
Aligned sources F	$B \times 36 \times 8 \times 3$; batch, time, source, feature	Five level, two rainfall and one release record joined along the source axis. Each keeps its own value, mask and age.
Embedded sources $H^{(0)}$	$B \times 36 \times 8 \times 32$	One token per source and time: type-specific projection, learned source and type vectors, location features and fixed time encoding.
Decoder memory M	$B \times 14 \times 32$; batch, token, feature	Eight last-step encoder summaries followed by six embedded plan records. Supplies the decoder's keys and values.
Station queries	$B \times 4 \times 32$; batch, station, feature	Final S_1–S_4 features plus the time encoding at $+6$ steps. Ask for levels at $t_n + 60$ minutes.
Training target Y	$B \times 4$; batch, main station	Actual levels at t_{n+6}, used as labels only once recorded. Missing labels are left out.
Forecast $\hat{Y}$	$B \times 4$; batch, main station	Levels at t_{n+6} in each station's documented units and reference height. Error bands from Section 1.4 are added afterwards.

Alignment and available information. We used only readings that were measured and received by t_n. A reading that arrives later counts as unavailable, even if it describes an earlier time. Each historical bin uses observations up to its end, never values interpolated from a later bin. When a level or release reading is missing, we carry the last value forward and let its age grow. If no earlier reading exists, we use the training median and set a missing flag. The mask marks a valid reading in that bin, and the age counts the time since the last valid reading. Filling a gap does not reset the age.

Rainfall needed different handling, because a rain record is an amount over an interval. We combined compatible readings into ten-minute totals without double counting, and we never carry rainfall forward. If a recorded interval cannot be split reliably, the affected bins are marked incomplete and given a neutral value, with their coverage and age kept. This keeps unknown rainfall separate from a measured zero. Units, gauge resets and duplicates are checked first.

Each source is scaled using training-period statistics. Water levels are scaled per station, because stations have different ranges and reference heights. Planned release rates use the same scaling as past releases. Availability flags stay separate from values, and forecasts are converted back to physical units for display.

1.4 Model computations and design justification

Notation and design. We chose factorized attention. Spatial attention mixes the eight sources at one time step, and temporal attention then mixes the 36 time steps of each source. The starting design has $L = 2$ encoder blocks, width 32 and four heads of width $d_k = 8$. History offsets are $i = -35, \ldots, 0$, and sources are $s = 1, \ldots, 8$. The equations below omit the batch axis. Each token starts as $x_{i,s} \in \mathbb{R}^3$: scaled value, availability and reading age. The main stations form the set $\mathcal{S} = \{S_1, S_2, S_3, S_4\}$.

The encoder keeps both the source and time axes until the end. This lets an earlier upstream change reach a downstream forecast, without pretending that unevenly spaced stations sit on a regular grid. River connectivity enters only as metadata and a learned bias; we used no graph layers or adjacency masks.

Embedding: what, where and when. Each starting token is

$$h_{i,s}^{(0)} = W_{\tau(s)} x_{i,s} + b_{\tau(s)} + e_s + e_{\tau(s)} + \phi(a_s) + \mathrm{PE}(i) \in \mathbb{R}^{32}. \tag{1}$$

Here $\tau(s)$ is the measurement type. Level stations share one 3-to-32 linear projection, rain gauges share another, and actual releases have their own. The learned vectors e_s and $e_{\tau(s)}$ identify the source and its type. A small feed-forward network ϕ turns the static metadata a_s into 32 features, with branch codes embedded as categories. Coordinates and river positions are scaled first, and missing positions are flagged.

PE uses the offsets given after Figure Q1.1. It is fixed, while the source and type embeddings are learned. Reading age is a separate input, so a stale value carried into the latest bin is not mistaken for a fresh one.

The result is one token per source and time, with shape $B \times 36 \times 8 \times 32$. We considered three alternatives. Mixing all sources into one token per time would combine them before attention. One token per source would compress each full history first. Treating each source independently would ignore upstream levels. Keeping both axes suits the spatial and temporal blocks we chose.

Attention and its inputs. For one head, attention combines values according to how well queries match keys:

$$\mathrm{Att}(X, Y; D) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + D\right) V, \quad Q = XW_Q, \quad K = YW_K, \quad V = YW_V. \tag{2}$$

The softmax runs over the key tokens, and each projection maps 32 features to eight. Joining the four head outputs and projecting back to 32 gives $\mathrm{MHA}_D(X, Y)$. Each sublayer has its own parameters, which are shared across all the times or sources it processes.

Table Q1.2: Query and key/value inputs for each attention step.

Step	Query input X	Key/value input Y
Spatial	Eight source tokens at one time	The same eight tokens; river bias $D = G^{\mathrm{river}}$
Temporal	36 times of one source	The same 36 historical tokens; $D = 0$ by default
Decoder	Four final station features plus target PE	Memory M: eight historical summaries and six plan tokens; $D = 0$

River-aware relative position bias. For a query source s and a key source u, we defined a scaled along-river distance $\bar{d}_{su}$ and a direction code r_{su}. The code is $+1$ when u is upstream of s, -1 when it is downstream, and 0 for itself or for undefined pairs. Two flags, m_{su}^d and m_{su}^r, mark whether each value is known. This keeps an unknown distance, filled with zero, separate from a true zero distance. The supplied connectivity decides these relations:

$$G_{su}^{\mathrm{river},j} = g_j\left(\bar{d}_{su}, m_{su}^d, r_{su}, m_{su}^r\right), \qquad G^{\mathrm{river},j} \in \mathbb{R}^{8\times 8}. \tag{3}$$

Each head's g_j is a small 4-to-8-to-1 network, shared across encoder blocks. Its output only shifts the attention weights; it never removes a source. Rain gauges and pairs without a river path are flagged

as unknown, but their location embeddings remain. The model can therefore treat an upstream station differently from a downstream one at the same distance. Distance guides the weights, but we did not turn it into a fixed travel delay.

Residual, normalisation and feed-forward update. Spatial, temporal and decoder attention all use the same update:

$$\tilde{X} = \mathrm{LN}(X + \mathrm{MHA}_D(X, Y)), \tag{4}$$

$$\mathcal{B}_D(X, Y) = \mathrm{LN}\Big(\tilde{X} + \mathrm{GELU}(\tilde{X}W_1 + b_1)W_2 + b_2\Big). \tag{5}$$

The additions keep the incoming features. LayerNorm (LN) works within each token, not across sensors or times. The feed-forward part widens 32 features to 64 and returns to 32. Spatial, temporal and decoder blocks each have their own parameters.

Spatial–temporal composition and reach. In encoder block $k = 1, \ldots, L$, the spatial step fixes the time and the temporal step fixes the source:

$$\tilde{H}^{(k)}_{i,:} = \mathcal{B}_{G^{\mathrm{river}}}(H^{(k-1)}_{i,:}, H^{(k-1)}_{i,:}), \qquad H^{(k)}_{:,s} = \mathcal{B}_0(\tilde{H}^{(k)}_{:,s}, \tilde{H}^{(k)}_{:,s}). \tag{6}$$

For computation, the spatial step reshapes to $(B \cdot 36) \times 8 \times 32$ and the temporal step to $(B \cdot 8) \times 36 \times 32$. Both restore the four axes afterwards. All 36 readings are already available at issue time, so temporal attention may look across the whole window. We would compare this with a causal mask during validation. Missing tokens keep their filled values, flags and ages, so they do not disappear during an outage.

One spatial–temporal pair already connects every final station to every observed source and time. For example, spatial attention can carry an earlier S_1 rise into the features of S_3, and temporal attention can then link it to recent S_3 readings. The second pair adds another learned transformation rather than more reach. Validation would compare one pair with two, and spatial-first with temporal-first order.

The companion notebook supported this argument about access to information. Its paired local histories had different targets, yet the temporal-only model gave both the same prediction. In the relay task, three local ST blocks reached a mean MAE near 0.007 within reach but 0.500 beyond it, while joint attention stayed near 0.0011. These are means over the three bundled runs. They motivated giving every station access to distant sources, but they did not decide the depth of our model. Head roles also varied between runs, so we would inspect several heads.

Future controls and decoder memory. Each known plan record p_h holds a release rate, plan coverage and lead time. Because it means something different from a past reading, it has its own projection:

$$P_h = W_{\mathrm{plan}} p_h + b_{\mathrm{plan}} + e_U + e_{\mathrm{planned}} + \mathrm{PE}(h), \qquad h = 1, \ldots, 6. \tag{7}$$

After the final encoder block, the last-step summaries and plan tokens form the decoder memory:

$$M = \mathrm{concat}_{\mathrm{tokens}}\left(H^{(L)}_{0,:}, P_{1:6}\right) \in \mathbb{R}^{14 \times 32}. \tag{8}$$

M therefore holds eight historical summaries and six future-plan records. Only schedules announced by issue time enter this branch; Section 1.5 covers gaps and revisions.

Each main station then asks for its level six steps ahead:

$$\mathbf{q}_s = h^{(L)}_{0,s} + \mathrm{PE}(+6), \quad s \in \mathcal{S}, \qquad Z = \mathcal{B}_0(Q_{\mathrm{station}}, M) \in \mathbb{R}^{4 \times 32}. \tag{9}$$

Here Q_{station} stacks the four queries, and each row z_s of Z is that station's decoded representation. The queries come from the final station features, while the keys and values come from M. The decoder predicts the one-hour target directly. It does not feed predictions back or produce intermediate steps.

Forecast mapping. A shared output head predicts each station's change from its latest level:

$$\delta_s = \mathrm{MLP}_{\mathrm{out}}(z_s), \qquad \hat{y}_s = \tilde{y}_{s,\mathrm{last}} + \delta_s. \tag{10}$$

The latest level $\tilde{y}_{s,\mathrm{last}}$ is in station-scaled units; if no reading exists, we use the training median. Predicting a change rather than the level itself makes persistence the natural starting point. The output has shape $B \times 4$ and is converted back to physical units.

Training objective. Let $\mathcal{O}$ be the issue–station pairs that have a recorded target, and let $\tilde{y}_{n+6,s}$ be the scaled level one hour after issue. We trained with mean absolute error (MAE):

$$\mathcal{L} = \frac{1}{|\mathcal{O}|} \sum_{(n,s)\in\mathcal{O}} |\tilde{y}_{n+6,s} - \hat{y}_{n,s}| \,. \tag{11}$$

We chose MAE because it is simple, robust to occasional sensor spikes, and reported in the same units as the forecast error. Missing targets are left out, and station scaling stops wide-range stations from dominating. Training keeps all flood events and adds plausible sensor outages and delays, so the model learns to use the masks.

Uncertainty bands. The model gives one level, but decision-makers also need to know how far to trust it. After training, we would collect the validation errors, actual minus forecast, for each station. We would group them by condition: normal flow or high water, and fresh or stale readings. The 10th and 90th percentiles of each group give a band that is added to the forecast. We would then check on the test period that about 80% of actual levels fall inside these bands. This needs no change to the model. A possible enhancement is to predict the band directly, with lower and upper outputs trained by pinball loss, if these simple bands prove too coarse.

Validation and evaluation. We would split the records by time: 60% for training, 20% for validation and 20% for testing. Each storm stays within one split, and windows that cross a boundary are dropped. Validation chooses the history length, depth, block order, masking, regularisation and fallback limits under one training budget. It also compares the river bias with no bias. When gains are unclear, we prefer the smaller design. The test period is used once, with plans and sensor arrivals rebuilt as they were known at each issue time.

We would compare the model with two baselines: persistence, and a temporal-only model that sees each station's own history plus the same rain and release inputs. We would report MAE and RMSE per station, errors during high water and errors by reading age. Band coverage and width would be checked in normal conditions, storms and outages. Removing spatial mixing or the future plans would show what each one adds.

We would also run paired checks, following the notebook. We hold a station's own history fixed and change a plausible upstream input or an announced plan. Other checks remove rain or tributary readings, or delay an upstream reading, while keeping timestamps consistent. Responses are judged by station and timing, because a late release need not reach every station within the hour. If a forecast ignores an input that clearly matters, we would check the data pipeline, weights and training before adding depth.

In this design, upstream levels, T_1, rainfall and past releases all reach each forecast through source and time attention. Announced releases have an extra path through the decoder. The unmonitored tributary has no reading, so only rainfall and later downstream rises hint at its inflow.

1.5 Operation and exceptional conditions

Every-ten-minute service. At each issue time, the service saves a snapshot of the inputs, aligns and scales the readings, and rebuilds the known release plan. One forward pass returns the four forecasts in physical units, with their bands, issue and target times, and quality flags. Input and model versions are archived for later review. The authority, not the model, sets the alert thresholds.

(i) Missing observations. Missing readings are filled, flagged and aged as in Section 1.3, so other sensors can partly make up for them. If a station's latest level is older than the one-hour limit in A8, the service publishes a labelled fallback: the last reading, or the training median if none exists. Bands are shown only for conditions whose coverage was checked during outages. Severe outages notify the operator and carry a "no reliable automated forecast" flag.

(ii) Delayed observations. Only readings received by the cutoff enter the snapshot. A late reading can still update the next forecast's history, but published forecasts are not silently changed; any revision is stored separately. Readings with future or inconsistent timestamps are set aside for review. We would also track errors by the actual age of each reading.

(iii) Announced future releases. The service uses the latest plan announced by issue time, including any cancellations. Each schedule is converted to an average rate for the six future bins, and gaps are flagged. An uncovered bin holds the latest actual rate. If that is also missing, it uses a flagged training reference value.

(iv) Scheduled versus actual releases. When actual release readings arrive, they update the release history for the next forecast. The original plan and its deviation are archived. Because training uses plans as they were announced, ordinary differences between planned and actual releases already show up in the validation errors, and therefore in the bands. Large deviations or policy changes raise a control-quality flag. When useful, forecasts under the revised plan and under the held last rate can be shown side by side.

Feasibility. The starting two-block encoder computes 101,376 attention scores per forecast, and the four decoder queries add 224. One block alone would need 50,688. The river bias adds only 196 parameters, and its output can be cached because it never changes. A model of this size should run comfortably on a CPU within the ten-minute cycle. Before deployment, we would still test data loading, inference, missed-cycle alerts and fallback publication.

1.6 Uncertainty, limitations, and additional information

Interpretation. The bands aim to contain about 80% of actual levels at each station. They come from past errors, so they reflect the inputs and operating behaviour seen in the validation period. We would check coverage during floods and release deviations, widen the bands on validation data if needed, and recheck on the test period. Coverage at each station does not guarantee that all four stations fall inside their bands together.

Unmonitored tributary. Unseen inflow can push forecasts too low at S_3 and S_4. Rainfall gives only a hint, and a rise at S_3 helps once it is observed. We would check band coverage during floods and flag unexplained rises. Sudden unseen inflows remain unresolved.

Changing travel times and backwater. Attention can pick different past times as conditions change, but it may fail under conditions it has not seen. Longer travel times may need more history. Spatial attention can also learn upstream associations, including possible backwater effects, but these need evidence. We would review the model when operating conditions change.

Future weather and rainfall gaps. Two gauges cannot describe future rain or every local storm. We would check errors and band widths during storms to measure how much these gaps cost.

Rare extremes, drift and faults. New flood records, changed reference heights, sensor fouling and new release policies can all cause errors. We would monitor data quality, errors and band coverage, use fallbacks, and review the model when patterns change. The system does not certify embankment safety or predict which areas will flood.

Table Q1.3: Additional information requests.

Additional request	How it helps	Usable without it?
Gauge on the unmonitored tributary, preferably measuring flow	Measures the missing inflow before it reaches downstream stations.	Yes, but flood warnings at S_3 and S_4 stay weaker.
Catchment maps, more rain gauges, radar or forecast rainfall	Links rain to runoff more clearly and gives earlier storm information. Weather forecasts should be archived as issued.	Yes, using the two existing gauges.
River profiles, rating curves, travel-time and runoff-delay bounds	Completes the distance test in A3 and shows whether backwater occurs. Could support physical routing later.	Yes, provisionally. The history window is revised if the test or validation fails.
Sensor arrival logs, quality flags, units and reference-height history	Rebuilds what was known at each issue time and helps detect clock and sensor changes.	Simulated delays replace missing logs. Live levels still need verified units and reference heights.
Versioned release plans, cancellations and actual-release readings	Shows plan changes and execution errors. Reservoir coordinates are also needed if missing.	Forecasting from past releases still works. Validating the future-plan branch needs these archives.
Longer flood records, policy changes, approved action thresholds	Tests rare floods and links forecasts to approved preparedness actions.	Yes, for forecasting. Rare-flood accuracy and action thresholds need further validation.

Sources and scope. Our sources were the AI651 Assignment 2 handout (Question 1, pp. 2–3), the Chapter 6 notes (Section 3.4(b)), and the executed `Question1.ipynb` with its bundled `checkpoints/manifest.json`. The notebook numbers come from toy tasks only. The river model itself has not been trained or benchmarked. Attention weights show associations, not causes, and the band coverage and release scenarios still need validation. We imposed no hydraulic water-balance constraint.

2 Question 2: Motion or Memory?

Final model: symmetrised D. All requested Tables 1–6, Figures 1–6 and answers Q2.1–Q2.17 follow.

Conventions: learned table entries are mean (seed 0, seed 1); deterministic references are single values. All prediction figures, GIFs and gap histograms use seed 0. Small differences are interpreted cautiously relative to the two-seed spread; no significance test is claimed. Visual descriptions report observed behavior, while explanations of internal mechanisms remain interpretations.

2.1 Setup and data analysis

Moving MNIST was loaded with `torchvision.datasets.MovingMNIST(split=None, download=True)`. Video indices 0-7999, 8000-8999 and 9000-9999 form training, validation and test sets. Frames are float32 in $[0, 1]$. Unless specified otherwise, performance uses all 1,000 test videos at start frame 1. Models see five frames and always emit five predictions; nonexistent targets are masked. Cross-speed comparisons use only horizons 1-2. Table 1 uses synthetic tensors; Figure 1 uses only training videos.

Table 1: Transformation and window checks.

Check	Expected	Yours
(a) $M(M(x))$ vs x	0; equal=True	0; equal=True
(b) $R^4(x)$ vs x	0; equal=True	0; equal=True
(c) $MRM(x)$ vs $R^{-1}(x)$	0; equal=True	0; equal=True
(d) $M(p)$ bright pixel	$(7, 10, 43)$	$(7, 10, 43)$
(e) $R(p)$ bright pixel	$(7, 43, 10)$	$(7, 43, 10)$
(f) $1\times$ input / target frames	$[1, 2, 3, 4, 5]$ / $[6, 7, 8, 9, 10]$	$[1, 2, 3, 4, 5]$ / $[6, 7, 8, 9, 10]$
(g) $2\times$ input / target frames	$[1, 3, 5, 7, 9]$ / $[11, 13, 15, 17, 19]$	$[1, 3, 5, 7, 9]$ / $[11, 13, 15, 17, 19]$
(h) $3\times$ input / target frames	$[1, 4, 7, 10, 13]$ / $[16, 19]$	$[1, 4, 7, 10, 13]$ / $[16, 19]$

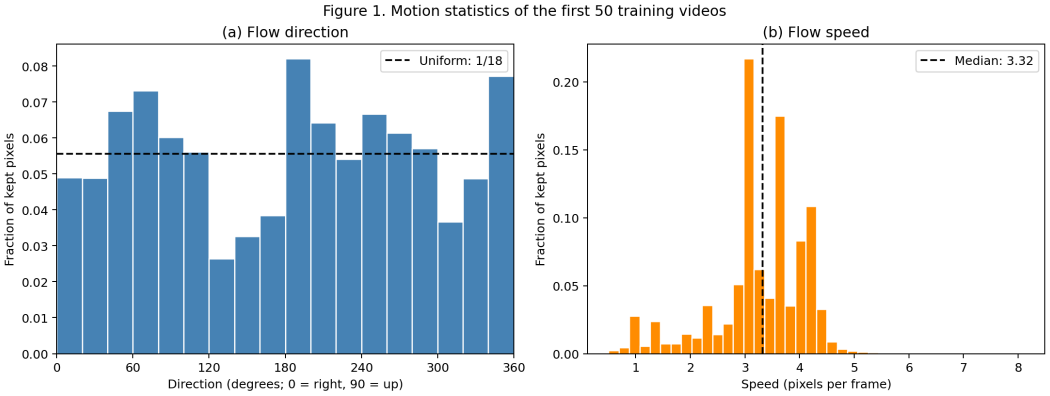


Figure 1: Farneback flow from training videos 0-49 over all 950 adjacent-frame pairs; 103,050 pixel observations retained. Both intensities exceed 0.2 and speed exceeds 0.5 pixels/frame. Median speed is 3.32 pixels/frame. Bars show fractions of kept observations; direction uses 18 bins of 20 degrees and a $\frac{1}{18}$ reference line.

Q2.1

Yes, mostly. Every $20°$ direction bin in Figure 1 contained motion, with shares from 0.0263 to 0.0820 against a uniform share of 0.0556. Mirroring maps a direction θ to $180° - \theta$, and rotation maps it to $\theta + 90°$, modulo $360°$. Neither created a direction the training data had never shown, but they did change how often each direction appeared, with histogram distances of 0.132 for mirror and 0.200 for rotation.

The median flow speed was 3.32 pixels per frame. At $2\times$ and $3\times$, that became roughly 6.64 and 9.95 pixels between observed frames. These were only rough references, since bounces change the net displacement. The flow estimate also had weaknesses: overlapping digits can corrupt it, and requiring a bright pixel in both frames may miss fast strokes. We did not measure how large these effects were, so the histogram supported broad familiarity with all directions, not exact symmetry of the motion.

Q2.2

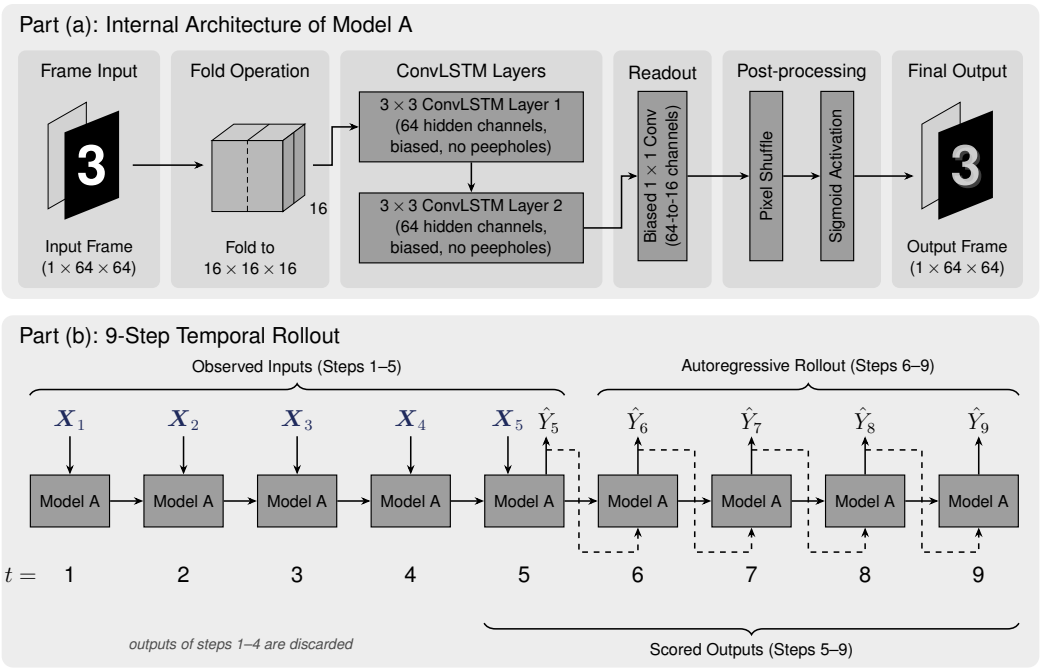
Mirror and rotation changed how the digits looked, while speed-up changed how far they moved between frames. The table below separates these effects.

Transformation	Motion statistics	Appearance
Mirror	Reflects direction; preserves speed. Frequencies can change.	Reflects strokes; keeps upright axis.
90° rotation	Rotates direction; preserves speed. Frequencies can change.	Turns upright strokes sideways.
Speed-up	Larger interval between observations; changes displacement.	Retained frames are not spatially transformed.

This made a controlled experiment possible. Section 2.4 tested the split with three more models. B and C each changed one thing in the baseline's training, and D changed the architecture relative to C:

Model	What changes	What it tests
B	Baseline ConvLSTM; training clips randomly mirrored or rotated, including the identity, at $1\times$.	Is the mirror/rotation failure about unfamiliar digit orientations?
C	Baseline ConvLSTM; training clips sampled at random speeds $1\times$, $2\times$ or $3\times$.	Is the speed failure about unfamiliar motion?
D	Same speed-varied clips as C; a network with larger reach (16 vs 8 pixels per axis per step).	Is the remaining speed failure about how far the network can pass information?

2.2 Baseline: ConvLSTM



Model A architecture: (a) Model A processes each frame through the two ConvLSTM layers in sequence; pixel shuffle (factor 4) is followed by sigmoid. (b) The diagram indexes predictions by the step producing them: $\hat{Y}_t = \hat{x}_{t+1}$. Thus outputs labelled $\hat{Y}_5$ through $\hat{Y}_9$ predict frames 6-10; steps 6-9 take the previous predicted frame as input. Horizontal arrows carry both layers' recurrent hidden and cell states; dashed arrows carry predicted frames without detaching gradients. All blocks share weights, and all recurrent states start at zero for each video.

No teacher forcing, dropout, normalisation or extra layers are used for A-C. All models use Adam at 10^{-3}, batch 32, 30 epochs, seeds 0 and 1, no schedule, one randomly sampled window per training video per epoch and final-epoch weights. Loss averages pixels/available horizons within a video before averaging videos equally. Validation monitors training; test results are reserved for evaluation and analysis.

Hardware: NVIDIA Tesla T4; PyTorch 2.11.0+cu130 on Linux. Training time is the mean training-pass wall time over all 30 epochs, including sampling/loading/transfers but excluding the one-time download, validation passes and checkpoint writing. Inference sums synchronised prediction time over all 1,000 original test videos at batch 32 under eval/no_grad, beginning after each batch is on the device. Wrapped timing includes all eight transforms, passes, inverse transforms and averaging.

Table 2: Mean (seed 0, seed 1); deterministic references are single values.

Model	MSE_1	MSE_2	MSE_3	MSE_4	MSE_5	$E^{(5)}$ $1\times$	Parameters
A	0.0170 (0.0172, 0.0169)	0.0205 (0.0206, 0.0204)	0.0228 (0.0229, 0.0226)	0.0246 (0.0247, 0.0244)	0.0262 (0.0264, 0.0260)	0.0222 (0.0224, 0.0220)	480784 (480784, 480784)
Blank	0.0426	0.0426	0.0426	0.0425	0.0425	0.0426	0
Copy-last	0.0492	0.0604	0.0669	0.0708	0.0732	0.0641	0

Figure 2: model A, 1x; overlap-screened video 9004

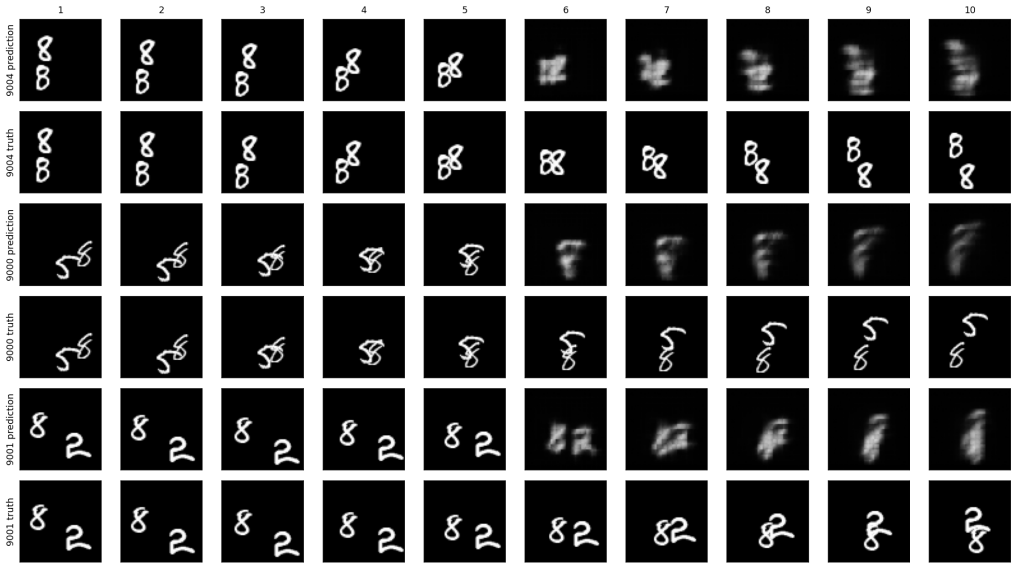


Figure 2: A, seed 0, $1\times$, test videos 9004, 9000 and 9001 (top to bottom). Each pair shows input/prediction above input/truth; columns 1-5 are observations and 6-10 are future frames. Video 9004 visibly contains overlapping 8s that subsequently separate in the truth.

Q2.3

Model A could move information 8 pixels along either axis in one step. Each frame was folded into a 16×16 grid, so one grid cell covered 4 original pixels, and each 3×3 gate convolution moved information by one cell. The longest path started at the new frame or the previous layer-1 hidden state and passed through both convolutions. That gave two cells, or $2 \times 4 = 8$ pixels, while the previous layer-2 state passed through only one convolution. The 1×1 readout and the pixel shuffle steps added no reach, and recurrence did not help either, because it only carried information on to the next step.

Since 8 pixels was the reach along each axis, the reachable region was a square. Its corner, at offset $(8, 8)$, was 11.3 pixels away in a straight line, so a speed had to be read together with its direction. Reach also only showed which paths existed, not whether the trained weights actually used them to track motion.

Q2.4

A beat both references on original clips. Its mean $E^{(5)}$ was 0.0222, compared with 0.0426 for Blank and 0.0641 for Copy-last (Table 2), which was 47.8% and 65.3% lower. Each seed beat both references on every one of the 1,000 test videos. Error grew with the horizon, from 0.0170 to 0.0262 (about 54%), because each prediction was fed back as the next input and mistakes built up.

Figure 2 showed the weakness. A moved blurred blobs in roughly the right direction, but lost the shapes of the digits. In video 9004, the two overlapping 8s separated in the truth, while A kept them as one smear. Videos 9000 and 9001 also lost the separate paths. MSE training explained the blur: when the future was uncertain, a faded average of the possible outcomes gave a lower loss than one sharp guess.

Q2.5

> I predict that model A will not be exactly consistent under either mirror or rotation, with rotation likely causing a larger accuracy loss because it changes digit appearance more strongly. Figure 1 shows motion in every direction bin, although unequal frequencies and flow bias prevent assuming exact motion symmetry. I expect 2x to degrade prediction and 3x to be particularly difficult: the median displacement references are 6.64 and 9.95 pixels per observed interval, while the longest one-step path in A spans 8 original pixels; reach is a bound, not a guarantee of reliable tracking.

Q2.5 is reproduced verbatim from commit `c3910e60c536719be346ddf9a05d1f54d4848271` (28 Oct 2026, branch `q2-predictions`, file `predictions/Q2_5.md`). A verbatim copy, added later when the code was moved to this repository, is at `assignment_02/Question2/predictions/Q2_5.md` on `main`. Retrospective assessment: the predicted ordering was broadly correct, but the early mean-error failure at $2\times$ was not anticipated by the reach intuition. This assessment does not modify the original prediction.

2.3 Testing the baseline

$E^{(5)}$ averages the five horizon MSEs and $E^{(2)}$ the first two, each over 1,000 test videos. For $T \in \{M, R\}$, C_T = MSE(f(TX), T $f(X)$); E_T = MSE(f(TX), T Y), averaging five horizons and all pixels per video, then videos.

Table 3: Mean (seed 0, seed 1); deterministic references are single values.

Model	$E^{(5)}$ $1\times$	C_M	E_M	C_R	E_R
A	0.0222 (0.0224, 0.0220)	0.00197 (0.00201, 0.00193)	0.0229 (0.0230, 0.0228)	0.00296 (0.00315, 0.00277)	0.0239 (0.0240, 0.0238)
Blank	0.0426	0	0.0426	0	0.0426
Copy-last	0.0641	0	0.0641	0	0.0641

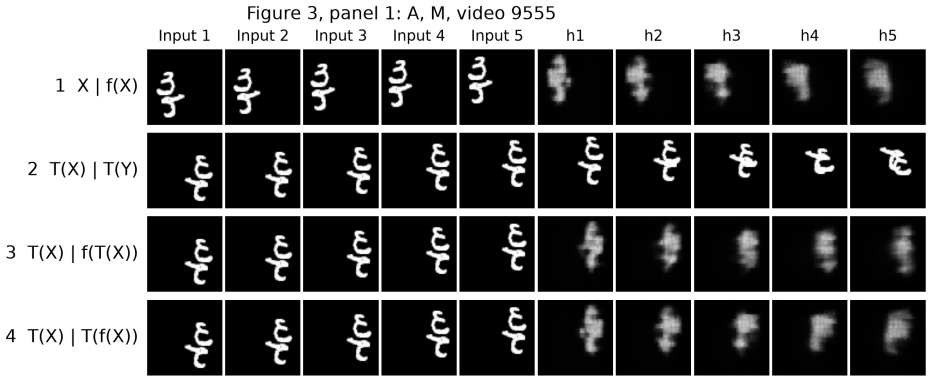


Figure 3(a): A seed 0, M, test video 9555, $C_M = 0.00882$. Rows: X/$f(X)$; $T(X)$/$T(Y)$; $T(X)$/$f(T(X))$; $T(X)$/$T(f(X))$. Inputs occupy columns 1-5, futures 6-10. GIF: `assignment_02/Question2/gifs/figure_3_1_M_video9555.gif`

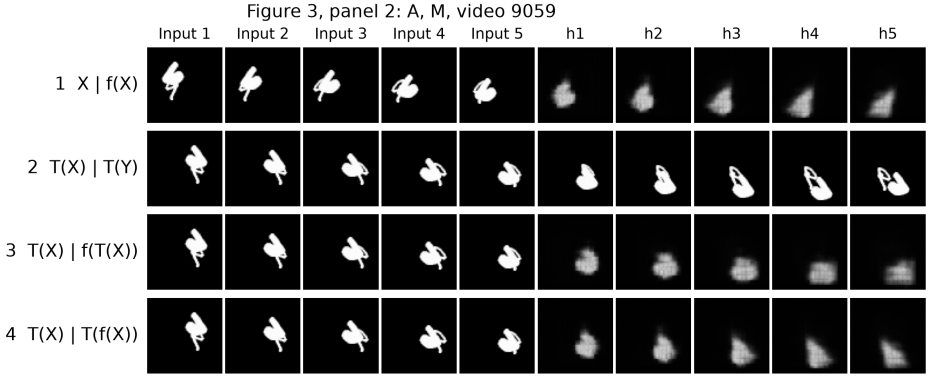


Figure 3(b): A seed 0, M, test video 9059, $C_M = 0.00726$. Rows: X/$f(X)$; $T(X)$/$T(Y)$; $T(X)$/$f(T(X))$; $T(X)$/$T(f(X))$. Inputs occupy columns 1-5, futures 6-10. GIF: `assignment_02/Question2/gifs/figure_3_2_M_video9059.gif`

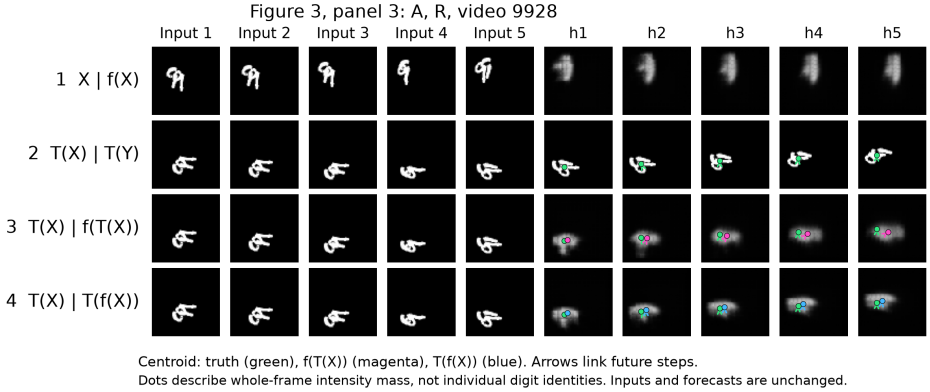


Figure 3(c): A seed 0, R, test video 9928, $C_R = 0.00902$. Rows: X/$f(X)$; $T(X)$/$T(Y)$; $T(X)$/$f(T(X))$; $T(X)$/$T(f(X))$. Inputs occupy columns 1-5, futures 6-10. Green truth, magenta row-3 and blue row-4 centroid dots/arrows highlight the motion disagreement; GIF is unannotated. GIF: `assignment_02/Question2/gifs/figure_3_3_R_video9928.gif`

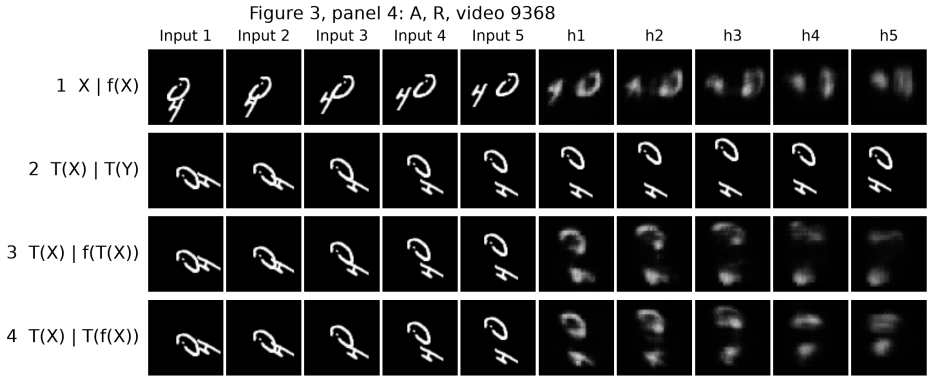


Figure 3(d): A seed 0, R, test video 9368, $C_R = 0.00789$. Rows: X/$f(X)$; $T(X)$/$T(Y)$; $T(X)$/$f(T(X))$; $T(X)$/$T(f(X))$. Inputs occupy columns 1-5, futures 6-10. GIF: `assignment_02/Question2/gifs/figure_3_4_R_video9368.gif`

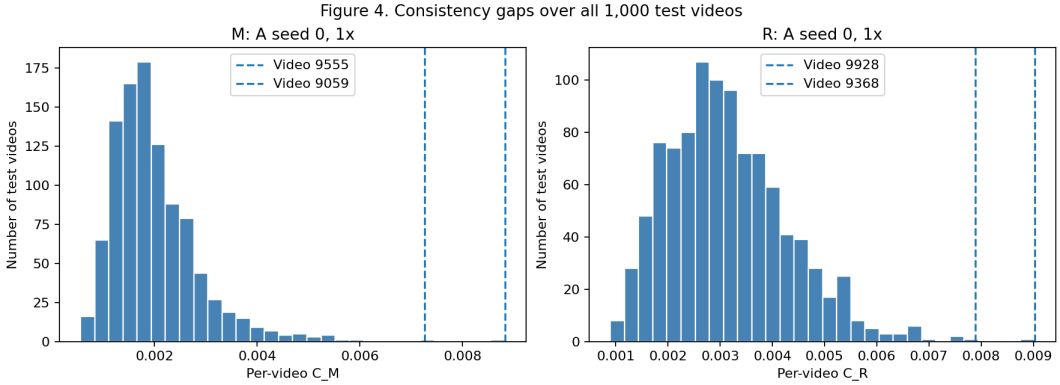


Figure 4: Per-video mirror and rotation consistency gaps for A seed 0 over all 1,000 test videos. The two largest mirror cases $(9555, 9059)$ and rotation cases $(9928, 9368)$ from Figure 3 are marked.

Q2.6

A zero consistency gap did not mean a correct forecast. Blank always returned an empty frame, and an empty frame looks the same after any mirror or rotation. Copy-last repeated the last input, and flipping that frame before or after copying gave the same result. Both therefore had $C_M = C_R = 0$, yet their transformed errors were 0.0426 and 0.0641, far worse than A's (Table 3). C_T only checked whether two prediction paths agreed with each other, while E_T checked whether the prediction matched the true future. Both were needed.

Q2.7

Video 9928 showed the clearest disagreement (Figure 3(c)) and had the largest rotation gap in seed 0, with $C_R = 0.00902$. In the truth, the digit moved mainly upward and slightly to the left. Row 3, $f(R(X))$, drifted right towards the centre as it rose. Row 4, $R(f(X))$, rose along the left side, closer to the true path. So rotating the input changed the model's account of the horizontal motion, and the coloured centroid dots and arrows made this visible across all five horizons. Both rows also lost the digit strokes and became blurred blobs.

Q2.8

Yes. Rotation was harder for A, and appearance was the stronger explanation. A's transformed error was 0.0239 under rotation and 0.0229 under mirror, and its consistency gap was also larger, 0.00296 against 0.00197 (Table 3). Q2.1 showed that both transforms produced directions A had already seen, so unfamiliar motion was unlikely. Q2.2 pointed to appearance instead: a mirror kept the digits upright, while a rotation turned them sideways, which A had never seen. Model B confirmed this.

After training on mirrored and rotated digits, its E_M, E_R and $E^{(5)}$ all rounded to 0.0226 (Table 5). One caveat remained: rotation also changed direction frequencies more than mirror did (0.200 against 0.132), so appearance and motion frequency were not fully separated.

Table 4: Mean (seed 0, seed 1); deterministic references are single values.

Model	$E^{(2)}$ $1\times$	$E^{(2)}$ $2\times$	$E^{(2)}$ $3\times$	$E^{(5)}$ $1\times$	$E^{(5)}$ $2\times$
A	0.0188 (0.0189, 0.0186)	0.0441 (0.0445, 0.0438)	0.0514 (0.0525, 0.0502)	0.0222 (0.0224, 0.0220)	0.0497 (0.0502, 0.0491)
Blank	0.0426	0.0425	0.0426	0.0426	0.0425
Copy-last	0.0548	0.0661	0.0703	0.0641	0.0719

Figure 5: video 9026, A

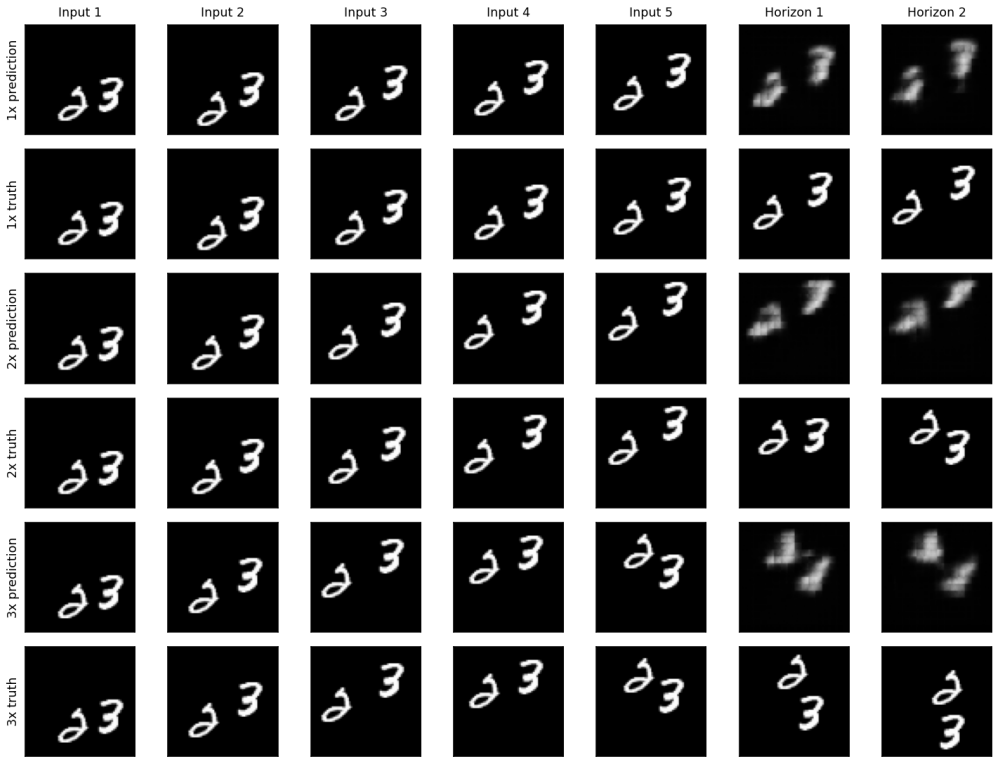


Figure 5: A seed 0, test video 9026. At each speed the upper row contains five inputs and two predictions; the lower row repeats the inputs and gives the two true futures. All displayed true frames have separated digits. The same video and windows are retained in Figure 6(b).

Q2.9

A broke down earlier than expected and was already worse than predicting nothing at $2\times$. Its $E^{(2)}$ rose from 0.0188 at $1\times$ to 0.0441 at $2\times$ and 0.0514 at $3\times$, while Blank scored 0.0425 and 0.0426 (Table 4). A still beat Copy-last. The reach argument missed this early failure, because a $2\times$ jump was about 6.64 pixels, inside the 8-pixel reach. Having the paths was not enough; the filters also had to learn to use them for larger jumps, bounces and repeated feedback.

Figure 5 showed how it failed: at $2\times$, A left the 3 smeared against the top wall, while the true digit bounced downward. At $3\times$, the blobs lagged near their last input positions. This stalling suggested

that A had not learned to handle faster motion or bounces, while the blur and fading came from uncertain MSE forecasts. Clear duplication of digits was not the main failure.

Q2.10

No, these results alone could not separate the two causes. A had never seen fast motion and also had limited reach, so either could explain the failure. A two-step experiment could tell them apart. First, compare A with C, which used the same architecture but was trained on mixed speeds. Then compare C with D, which kept C's training but had a larger reach. A gain from A to C would point to unfamiliar motion, and a further gain from C to D would point to reach. If neither improved, the optimisation and the fixed training budget would need checking before calling the task impossible.

2.4 What Can Data Fix?

Q2.11

> After observing baseline A mean gaps CM=0.00197, CR=0.00296, and 3x E(2)=0.0514, I predict symmetry-augmented B will reduce orientation-related inconsistency and transformed error, without guaranteeing exact equivariance. I predict speed-augmented C will improve faster-motion prediction but may remain limited at 3x by its 8-pixel per-axis reach. I predict D, with the same speed augmentation and 16-pixel per-axis reach, will improve 3x tracking relative to C, while not guaranteeing mirror or rotation consistency.

Q2.11 is reproduced verbatim from commit `310dcbf1dcca27fdcb80b7b967058ac50ff3f2df` (28 Oct 2026, branch `q2-predictions`, file `predictions/Q2_11.md`). A verbatim copy, added later when the code was moved to this repository, is at `assignment_02/Question2/predictions/Q2_11.md` on `main`. Retrospective assessment: B and C broadly behaved as predicted; D did not outperform C at $3\times$. The saved record identifies this as the prediction before B-D training, not a replacement written after seeing their results.

B uses A's architecture and $1\times$ windows with a uniformly drawn element of $\mathcal{G}$ for each paired input/target. C and D draw speed uniformly from {1,2,3}, then start uniformly from the allowed starts: 1-14, 1-8 and 1-2 respectively. They use no symmetry augmentation. A and B draw starts 1-11. D changes only the architecture relative to C; optional D+B was not trained.

Table 5(a): Original and transformed errors. Mean (seed 0, seed 1); deterministic references are single values.

Model	$E^{(5)}$ $1\times$	C_M	E_M	C_R	E_R
A	0.0222 (0.0224, 0.0220)	0.00197 (0.00201, 0.00193)	0.0229 (0.0230, 0.0228)	0.00296 (0.00315, 0.00277)	0.0239 (0.0240, 0.0238)
B	0.0226 (0.0236, 0.0216)	0.00108 (9.60×10^{-4}, 0.00121)	0.0226 (0.0235, 0.0217)	0.00141 (0.00125, 0.00156)	0.0226 (0.0236, 0.0216)
C	0.0269 (0.0263, 0.0275)	0.00135 (0.00122, 0.00149)	0.0271 (0.0265, 0.0277)	0.00280 (0.00318, 0.00243)	0.0281 (0.0275, 0.0287)
D	0.0292 (0.0289, 0.0294)	5.33×10^{-4} (7.08×10^{-4}, 3.57×10^{-4})	0.0293 (0.0290, 0.0295)	0.00143 (0.00187, 9.87×10^{-4})	0.0298 (0.0296, 0.0299)

Table 5(b): Speed, size and timing. Mean (seed 0, seed 1); deterministic references are single values.

Model	$E^{(2)}$ $1\times$	$E^{(2)}$ $2\times$	$E^{(2)}$ $3\times$	Parameters	Train s/ epoch	Infer s/ 1000
A	0.0188 (0.0189, 0.0186)	0.0441 (0.0445, 0.0438)	0.0514 (0.0525, 0.0502)	480784 (480784, 480784)	12.3 (12.3, 12.3)	0.398 (0.398, 0.399)
B	0.0190 (0.0200, 0.0181)	0.0436 (0.0427, 0.0445)	0.0493 (0.0489, 0.0497)	480784 (480784, 480784)	12.6 (12.6, 12.7)	0.401 (0.399, 0.402)
C	0.0234 (0.0232, 0.0236)	0.0288 (0.0285, 0.0292)	0.0333 (0.0331, 0.0334)	480784 (480784, 480784)	12.2 (12.3, 12.2)	0.397 (0.398, 0.397)
D	0.0266 (0.0262, 0.0270)	0.0303 (0.0299, 0.0306)	0.0335 (0.0334, 0.0336)	480784 (480784, 480784)	45.3 (45.3, 45.2)	0.653 (0.653, 0.653)

Q2.12

D doubled A's reach to 16 pixels per axis without adding any parameters. Its first gate convolution used dilation 1, as in A, while its second used dilation 3, so its 3×3 filter looked at cells three apart. The longest path therefore spanned $1 + 3 = 4$ grid cells. At 4 pixels per cell, that was 16 pixels per axis, twice A's 8, and 22.6 pixels along the diagonal. Combining offsets $\{-1, 0, 1\}$ with $\{-3, 0, 3\}$ covered every offset from -4 to 4, so no cells were skipped. Everything else stayed the same: the two 64-channel layers, the readout, the nine-step rollout and the training setup. D had 480,784 parameters, below the 721,176 cap, and used no attention.

Q2.13

Both of A's failures came from what its training data lacked, not from its architecture. The mirror and rotation failure was appearance, and the speed failure was unfamiliar motion. We found no evidence for architectural reach.

Failure	Label	Table 5 comparison	Evidence
Mirror/rotation	Appearance	A vs B	C_M: 0.00197 to 0.00108; C_R: 0.00296 to 0.00141. E_M/E_R: 0.0229/0.0239 to 0.0226/0.0226.
Fast motion	Unfamiliar motion	A vs C	$2\times$ $E^{(2)}$: 0.0441 to 0.0288; $3\times$: 0.0514 to 0.0333 (35.3% lower). Same architecture and reach.
Residual fast-motion error	Not architectural reach (no evidence)	C vs D	$3\times$ $E^{(2)}$: 0.0333 vs 0.0335; D also worse at $1\times$ and $2\times$.

In our runs, C was not bad at $3\times$. Training on fast clips alone cut A's $3\times$ error by about a third, with the same 8-pixel reach, and D's doubled reach added nothing on top. Had it gone the other way, with C still failing and D succeeding, the cause would have been architectural reach. Both models were trained on the same clips, so reach would have been the only difference left. Extra data could not have closed that gap, because training adjusts the connections a network already has, but it cannot add a connection between cells that are too far apart. Eight pixels turned out to be enough for two reasons. A digit is not a single point, so some of its strokes were always within reach of where it moved next. Also, a 10-pixel diagonal jump is only about 7 pixels along each axis. D's result still deserves some caution, because its training loss had not settled by epoch 30, so the conclusion holds for this design and budget, not for larger reach in general.

Q2.14

Yes. Original $1\times$ $E^{(5)}$ rose from A's 0.0222 to B's 0.0226 (+1.78%), C's 0.0269 (+21.1%) and D's 0.0292 (+31.3%). B's small cost buys improved orientation robustness. C's cost is worthwhile when

faster-motion accuracy matters, given the gains in Q2.13. D is not a useful accuracy trade-off over C in these runs: it appeared worse at all three speeds and took 45.3 rather than 12.2 seconds per training epoch. It remained the underlying final model because the handout required D, not because it was the most accurate candidate.

2.5 The final model

The final predictor is symmetrised D, as required. Symmetrised B is included for comparison only. It is not substituted for the required final model. The wrapper needs no retraining, and each underlying parameter is counted once.

Table 6(a): Original and transformed errors. Mean (seed 0, seed 1); deterministic references are single values.

Model	$E^{(5)}$ $1\times$	C_M	E_M	C_R	E_R
Symmetrised B	0.0220 (0.0230, 0.0210)	4.24×10^{-17} (3.94×10^{-17}, 4.54×10^{-17})	0.0220 (0.0230, 0.0210)	4.32×10^{-17} (4.02×10^{-17}, 4.62×10^{-17})	0.0220 (0.0230, 0.0210)
Symmetrised D (final)	0.0290 (0.0287, 0.0294)	3.08×10^{-17} (3.41×10^{-17}, 2.75×10^{-17})	0.0290 (0.0287, 0.0294)	3.16×10^{-17} (3.48×10^{-17}, 2.83×10^{-17})	0.0290 (0.0287, 0.0294)

Table 6(b): Speed, size and timing. Mean (seed 0, seed 1); deterministic references are single values.

Model	$E^{(2)}$ $1\times$	$E^{(2)}$ $2\times$	$E^{(2)}$ $3\times$	Parameters	Train s/ epoch	Infer s/ 1000
Symmetrised B	0.0186 (0.0196, 0.0176)	0.0427 (0.0421, 0.0433)	0.0481 (0.0482, 0.0480)	480784 (480784, 480784)	12.6 (12.6, 12.7)	3.49 (3.48, 3.50)
Symmetrised D (final)	0.0266 (0.0261, 0.0270)	0.0302 (0.0299, 0.0305)	0.0334 (0.0332, 0.0335)	480784 (480784, 480784)	45.3 (45.3, 45.2)	5.72 (5.76, 5.69)

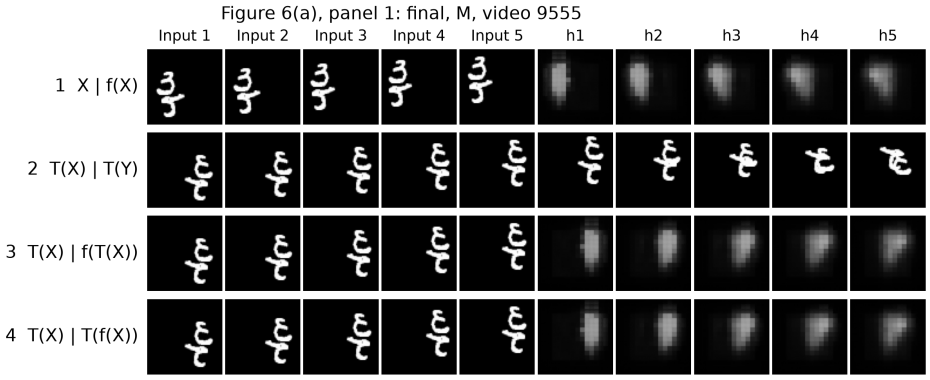


Figure 6(a), panel 1: Symmetrised D seed 0 on exactly Figure 3(a)'s video 9555 and transformation M. The same four rows and input/future columns are retained; horizontal labels are enlarged. GIF: `assignment_02/Question2/gifs/figure_6a_1_M_video9555.gif`

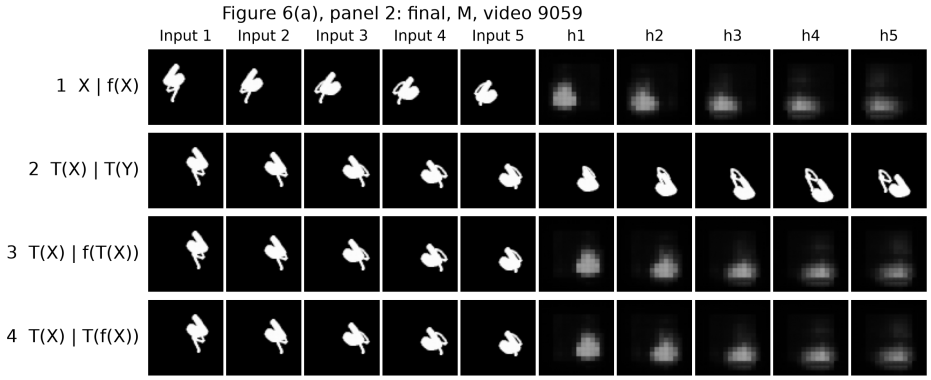


Figure 6(a), panel 2: Symmetrised D seed 0 on exactly Figure 3(b)'s video 9059 and transformation M. The same four rows and input/future columns are retained; horizontal labels are enlarged. GIF: `assignment_02/Question2/gifs/figure_6a_2_M_video9059.gif`

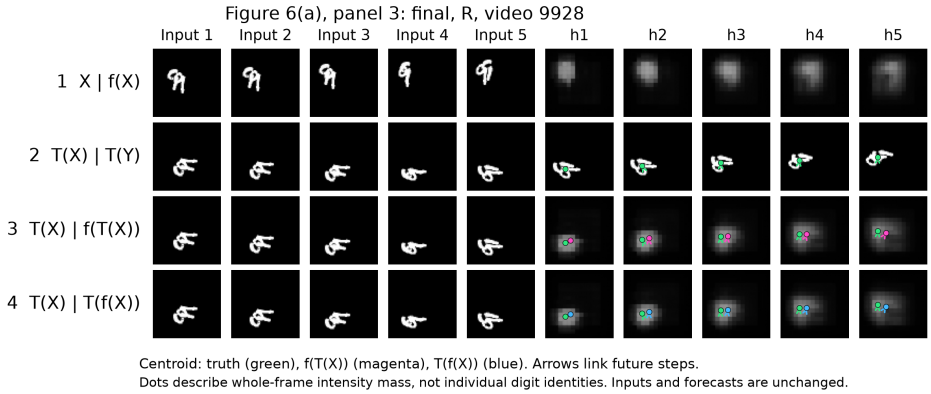


Figure 6(a), panel 3: Symmetrised D seed 0 on exactly Figure 3(c)'s video 9928 and transformation R. The same four rows and input/future columns are retained; horizontal labels are enlarged. Centroid markers in rows 3 and 4 now agree but remain displaced from the truth; GIF is unannotated. GIF: `assignment_02/Question2/gifs/figure_6a_3_R_video9928.gif`

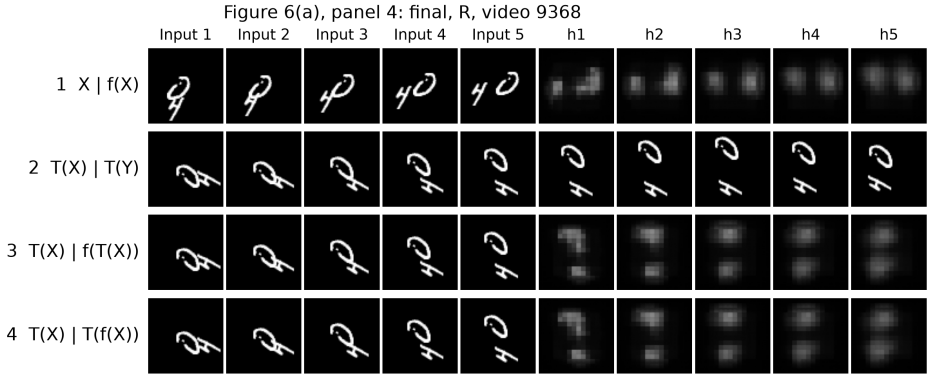


Figure 6(a), panel 4: Symmetrised D seed 0 on exactly Figure 3(d)'s video 9368 and transformation R. The same four rows and input/future columns are retained; horizontal labels are enlarged. GIF: `assignment_02/Question2/gifs/figure_6a_4_R_video9368.gif`

Figure 6(b): matched truth, A and final; test video 9026

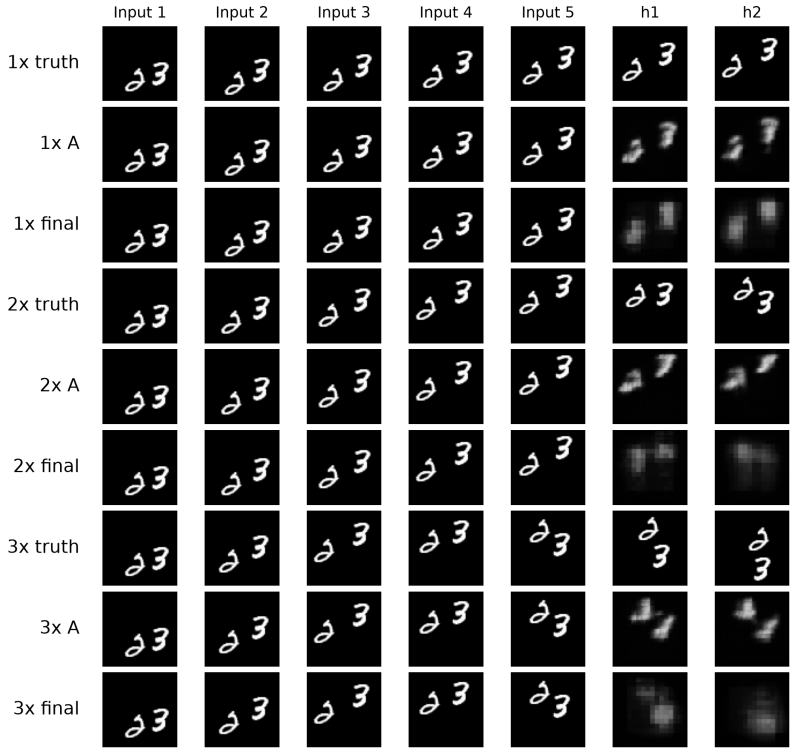


Figure 6(b): Matched comparison on Figure 5's video 9026. At each speed, the three rows show truth, A and final symmetrised D (seed 0). All repeat the same five inputs and display only the first two futures, so the baseline-to-final change can be read directly.

Q2.15

Built-in symmetry is exact; learned symmetry is approximate. B retained $C_M = 0.00108$ and $C_R = 0.00141$. For the wrapper, $\bar{f}(X) = \frac{1}{8}\sum_{g\in\mathcal{G}} g^{-1} f(gX)$. For fixed T, set $h = gT$, so $g = hT^{-1}$ and $g^{-1} = Th^{-1}$:

$$\bar{f}(TX) = \frac{1}{8}\sum_{h\in\mathcal{G}} T\,h^{-1} f(hX) = T\,\bar{f}(X).$$

The last step used the linearity of the pixel permutation and summed over the same group. Thus $C_M = C_R = 0$ in real arithmetic. Table 6 records floating-point residue: 4.24×10^{-17} / 4.32×10^{-17} for wrapped B, and 3.08×10^{-17} / 3.16×10^{-17} for wrapped D. The reported errors on mirrored or rotated clips are equal (up to rounding) to the error on the original clips.

Without any extra training, wrapping slightly improved original $E^{(5)}$: B 0.0226 to 0.0220 and D 0.0292 to 0.0290. Inference over 1,000 videos costed 3.49 as compared to 0.401 seconds for B ($8.71\times$), and 5.72 compared to 0.653 seconds for D ($8.76\times$). Averaging can hurt when the eight inverse-transformed forecasts disagree and blur a good prediction. It did not worsen mean MSE here, although the images remained blurred. The required final model is symmetrised D; optional D+B was not trained.

Q2.16

'D wrapped' fixes consistency for video 9928, but not the forecast. In Figure 6(a), panel 3, rows 3 and 4 are now identical. However, their annotated centroids remain right of the truth, and the digit strokes become blurred and smudged. The symmetrisation in 'D wrapped' removes the disagreement

identified in Q2.7, but it does not restore the correct trajectory or the identity of the digit. Figure 6(b) shows model A and the final model beside the same true futures at every speed. D's speed-augmented training helps the final model respond better to fast displacement and the bounce, but its predictions remain blurry. The controls in Q2.13 attribute the main speed gain to exposure, not extra reach; overlap identity remains unresolved.

Q2.17

Partially, its performance is mixed. It did learn some useful motion-based predictions; however, it failed to learn a reliable rule for tracking the individual digits. As we saw, model A beats Blank and Copy-last on every test video, while model C, which was trained on mixed speeds, beats A on the sped-up clips. Controlled comparisons in Q2.13 show the main sources of transformation and speed failures. The final model's predictions are consistent under mirror and rotation, but Figure 6 shows that consistent predictions can still be wrong. Symmetry alone does not prove correctness. It only ensures that the model behaves consistently under certain transformations. The evidence for a skeptic is a combination of true future errors, consistency gaps observed, both seeds and matched examples, rather than symmetry alone. A remaining failed test is recovering separate digit identities after overlap. Unseen accelerations and speeds beyond the tested range remain further challenges.

References

Xingjian Shi, Zhourong Chen, Hao Wang, Dit-Yan Yeung, Wai-kin Wong, and Wang-chun Woo. Convolutional LSTM network: A machine learning approach for precipitation nowcasting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

Nitish Srivastava, Elman Mansimov, and Ruslan Salakhutdinov. Unsupervised learning of video representations using LSTMs. In *International Conference on Machine Learning (ICML)*, 2015.

A Reproducibility and evidence (Question 2)

All results came from one Colab run of the code in `assignment_02/Question2/`, driven by the notebook `question_2.ipynb`. Checkpoints and raw per-video arrays are not included in the repository because of their size. All eight checkpoints reached epoch 30, and the per-video arrays reproduced the reported aggregates. All video IDs are global dataset indices. The matched panels in Figures 3 and 6 were re-rendered from the saved seed-0 weights with larger labels and centroid annotations; their context and truth regions were checked to be pixel-identical to the originals, and no metric changed.

B AI assistance disclosure

I used AI to help brainstorm, evaluate, and refine concepts across this assignment. In Question 1, I consulted AI to review the problem setup, discuss trade-offs between different modeling approaches, unpack the companion notebook, and proofread my written report. In Question 2, I used AI to clarify the problem requirements, transformations, reach / consistency and other related concepts. Furthermore, I used it to assist with code drafting, figure/diagram generation, and in my answer review and grammatical proofreading.